Operating Systems: Homework 1 (HW1)

Instructor: Dr. Probir Roy

Instructions

- No teamwork is allowed for all homework assignment.
- `VeriCite` option has been enabled on Canvas to compare your submission with other submissions from this semester and past semesters and Internet database.
- Your solution should be written in a ".docx", ".doc", or ".pdf" format and submitted to the assignment folder HW1 on Canvas.
- A subset of questions will be graded.

Questions

1. What are the benefits of virtualizing CPU and virtualizing memory?
2. The following questions are regarding `fork()` operation:

   (a.) When a parent process creates a child process using the `fork()` operation, is there any part of memory in principle shared between the parent process and the child process during their ongoing executions? Explain it.

   (b.) Identify the values of pid at Lines A, B, C, and D in the following program. (Assume that the actual pids of the parent and child are 2611 and 2612, respectively.)

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid1, pid2;

    pid1 = fork();

    if (pid1 < 0) {
        fprintf(stderr, "Fork Failed");
        return 1;
    } else if (pid1 == 0) {
        pid2 = getpid();
```

```
        printf("A: pid1 = %d\n", pid1);   /* A */
        printf("B: pid2 = %d\n", pid2);   /* B */
    } else {
        pid2 = getpid();
        printf("C: pid1 = %d\n", pid1);   /* C */
        printf("D: pid2 = %d\n", pid2);   /* D */
        wait(NULL);
    }
    return 0;
}
```

(c.) Explain what the output will be at Line A in the following program.

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int value = 150;

int main() {
    pid_t pid;
    pid = fork();

    if (pid == 0) {
        value += 50;
        return 0;
    } else if (pid > 0) {
        value -= 50;
        wait(NULL);
        printf("Value = %d\n", value);  /* A */
        return 0;
    }
}
```

(d.) Explain how the separation of `fork()` and `exec()` helps the shell implement some functions.

3. Both the software (OS) and the hardware work together to implement restricted operation. Explain how they work to achieve this goal.

4. Explain how the OS regains control of the CPU in a non-cooperative way in order to switch processes.

5. In Project P1, `exit()` is required in implementing `spin`. You can test the `spin` command again with `exit()` is commented out. What error do you get? Explain in details what the purpose of `exit()` is. Hint: `exit()` is eventually implemented in `proc.c`.

6. Five jobs $P_1, \ldots, P_5$ arrive at a processor at time 0, 1, 2, 3, 4 and the lengths of their CPU bursts are 8, 2, 5, 2, 3, respectively. For each of the following scheduling algorithms, draw the Gantt Chart and calculate the average waiting time and the average response time:

   - FCFS: First-Come-First-Served

- SJF: Shortest-Job-First
- SRTF: Shortest-Remaining-Time-First;
- P-PS: Preemptive priority scheduling with priority assignment (3, 2, 1, 4, 3) to $(P_1, \ldots, P_5)$, where priority 1 is the highest priority;
- RR: Round-Robin with a time quantum of 2.

7. Among the above five scheduling policies, list all that might result in starvation.

8. Consider a system running 10 I/O-bound processes and 2 CPU-bound process. Assume that the I/O-bound processes issue an I/O operation once for every millisecond of CPU computing and that each I/O operation takes 10 milliseconds to complete (we assume that there is no competition among I/O operations). Assume that the CPU-bound processes do not issue any I/O operations. A round-robin scheduler with quantum size 1 milliseconds is used to schedule all these 12 processes together. We assume that the context-switching overhead is 0.1 millisecond and that all processes are long-running tasks. Calculate the CPU utilization.

9. Multi-Level-Feedback-Queue (MLFQ) is one of CPU scheduling policies with some great features. As we discussed in class, it can be used to shoot four hawks which represent the problems in many scheduling policies. Briefly describe each of four hawks. Also briefly explain how MLFQ shoots each.

10. We consider MLFQ policy for all jobs in Question 6. We ignore priority boost and I/O and the detailed policy is as follows:

    - A new job enters Q_1 in Round-Robin (RR). When it gains CPU, job receives 1 time unit; If it does not finish in 1 time unit, it is moved to Q_2.
    - At Q_2, job is again served in RR and receives 2 time units. If it still does not complete, it is preempted and moved to Q_3.
    - Run RR in Q_3 with quantum size 4.

    Draw the Gantt Chart and calculate the average waiting time and the average response time.

11. We will use a simulator `mlfq.py` to see how MLFQ behaves. The simulator and README can be found on this page: `http://pages.cs.wisc.edu/~remzi/OSTEP/Homework/homework.html`. They are also stored in the Docker image from P1 under Folder `/simulation/HW-MLFQ` with Python 2.7 support. Test this command `mlfq.py -c -s 20191 -B 0 -M 0 -j 4 -n 3 -m 10` in the command line. Draw the Gantt Chart and obtain the waiting time for each job.